Cellular Evolution

Fabio Giorgio Miceli s332958, Adriano Valentini s319844

June 11, 2025

1 Introduction

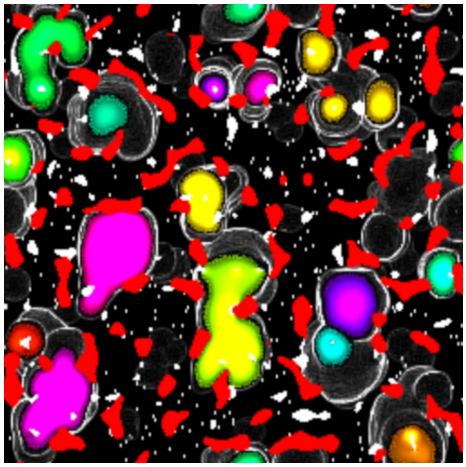


Figure 1: An image of a simulation, the red points are obstacles and the white are foods, other color rappresent the creatures

Our project is inspired by the **cellular automaton framework proposed by LENIA**, but we introduced several significant modifications, resulting in a fundamentally new system.

The core idea is similar to that of LENIA: to discover a **stable cell capable of surviving in a world with specific characteristics**. However, we approached the problem from a **machine learning perspective**. In our system, the cell starts off as a **single atomic entity** equipped with its own form of **intelligence**, represented by a **deep neural network**. The goal of this neural network is to enable the cell to **expand, gather as much energy as possible, or occupy the maximum amount of space** within the environment.

At each time step (epoch), the world is **randomly generated** based on certain **hyperparameters**, such as the distribution of food and obstacles. Within

this dynamic world, **N different creatures** are instantiated. These creatures must **compete with one another** to achieve the best possible performance according to the task.

The evolution of each cell is guided by a **reinforcement learning technique called Evolution Strategy**. However, we have **adapted and modified the standard method**. Typically, Evolution Strategy involves training N mutated versions of a base model in different environments. Their performance (fitness scores) determines how the original model's parameters are updated.

Instead, we experimented with a setup in which **all variations of the cell are placed in the same environment**. This forces the agents to **adapt to one another's presence**, leading to emergent competitive behavior where success depends not only on adapting to the environment but also on outperforming other agents within the same space.

To support the computational demands of this simulation, we implemented the system using **CUDA** and conducted our experiments on **two different NVIDIA GPUs**:

- NVIDIA 930MX
- NVIDIA A5500

This setup allowed us to explore both the performance scalability and the impact of different hardware capabilities on training and simulation fidelity.

2 World Rule

We defined several rules governing the world and the behavior of the cells. Some of these rules are fixed, while others can be modified through the use of hyperparameters.

World Representation

The world is modeled as a matrix of cells. Each cell contains the energy of the occupant cell, which indicates how much of that creature is present in that cell, each cell also has an ID, representing the owner of that map cell. In the case of an empty cell, it holds the amount of food available. Possible IDs range from -1 (representing obstacles) to N. A value of 0 indicates a free or neutral cell, whereas values greater than 0 indicate that the cell is occupied by the corresponding creature.

Additionally, each cell has a signaling value, which represents how cells communicate with each other. For making decisions, cells only have access to a local neighborhood around themselves containing signaling and energy values.

Cell Aging and Energy Decay

To make adaptation more challenging and encourage models to become more efficient, we introduced a decay or aging mechanism for the cells. At each step, a fixed amount of energy is subtracted from the cell. If the cell's energy falls below a certain threshold, the cell dies, its ID is reset to 0, and any leftover energy remains in that position as food.

Food and Energy Transfer

To ensure longer survival, we introduced food clusters and randomly distributed food across the map. Cells approaching food can "consume" it to increase their energy. When a food cell is attacked it spread its energy all around increasing the values of all cells nearby.

Obstacles

In addition to food, obstacles are present on the map as previously described. Obstacles are portions of the map whose energy cannot change and remain static throughout the epoch. Creatures cannot interact with obstacles in any way, any energy directed onto obstacles is lost.

Creature Interactions and Combat

Creatures can attack each other using a mechanism similar to consuming food. To determine the winner among contenders, two main formulas are used: one applies when the cell in question is empty and multiple different sources contribute energy to it, and another when the cell is already occupied by a creature but other creatures invest energy in that cell to kill it.

World Phases: Epochs and Steps

The world is divided into different phases: epochs, which define when the world and new creatures are generated, and steps, which are the phases in which living cells decide how to act in the world. Each epoch consists of n steps, after which the epoch ends. Another termination condition for an epoch is the death of all cells.

During each epoch, cumulative scores are computed and updated at every step, indicating how much energy each creature has possessed throughout the epoch and how many map cells they have controlled.

3 Program Steps

The program is divided into several phases, of which four main ones can be distinguished:

Info and Allocation Phase

This preparation phase involves unpacking the data required for the simulation, retrieving information about the current GPU, performing all necessary memory allocations for the simulation, and loading the model if it is available or generating it if it does not exist.

Epoch Preparation Phase

This phase consists of generating the world and the variations of the creatures (given the original model) to be used for training during the current epoch.

Step Phase

During this phase, the actions of the living cells of each creature are computed. The world state is updated based on the contributions of the creatures in each cell, the score vectors for each creature are updated, and the count and indices of living cells are recalculated.

Recombination Phase

At the end of each epoch, the recombination phase updates the parameters of the original model by considering the scores obtained by its variations, according to the formula:

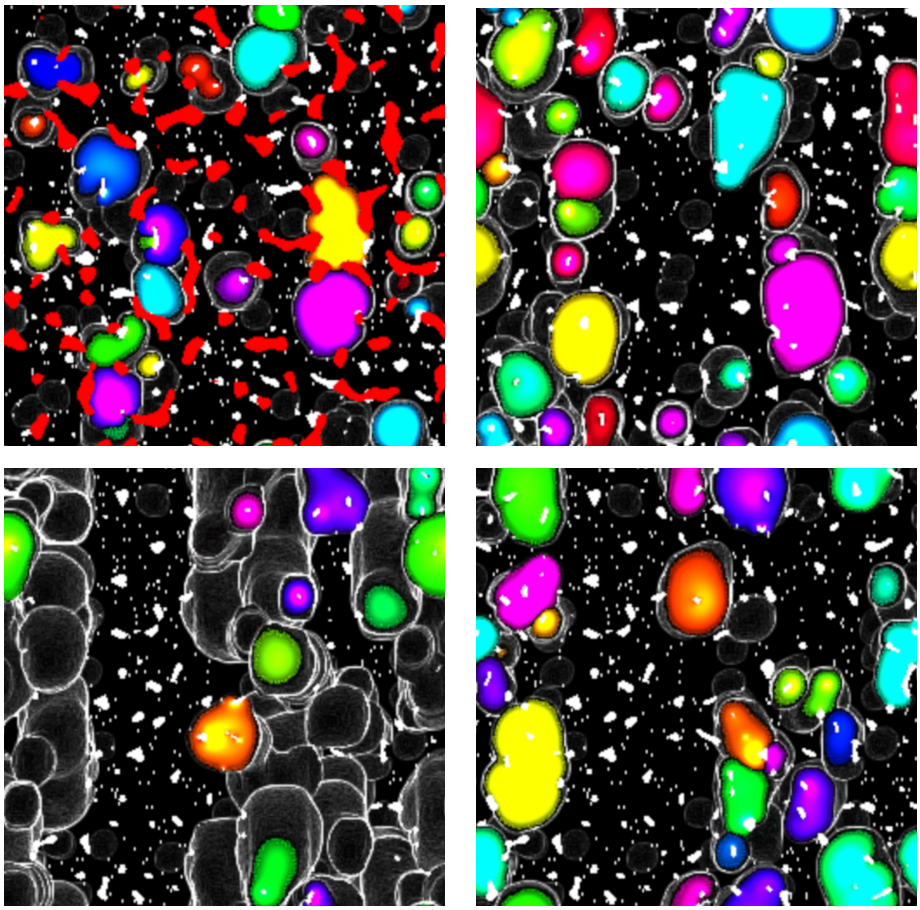
$$\theta_{t+1} = \theta_t + \alpha \cdot \frac{1}{n\sigma} \sum_{i=1}^{n} R_i \cdot \epsilon_i \tag{1}$$

- α **(Learning rate)**: Controls the magnitude of each parameter update.
- σ **(Perturbation standard deviation)**: Determines the spread of the Gaussian noise added to the parameters.
- n **(Population size)**: Number of sampled perturbations used to estimate the gradient.
- θ_t **(Parameter vector at iteration** t**)**: The current policy parameters being optimized.
- ϵ_i **(Noise sample)**: Gaussian noise independently sampled for each individual in the population.

- R_i **(Reward or fitness)**: The scalar reward received from the environment when using the perturbed policy $\theta_t + \sigma\epsilon_i$.
- θ_0 **(Initial parameters)**: The starting point of the policy parameter vector before optimization begins.

Cleanup Phase

This final phase, executed at the end of all epochs, is responsible for freeing all allocated memory resources on both the CPU and GPU.



4 Memory manage

The functioning of our program requires the use of different memory sections, both CPU and GPU.

A portion of the GPU memory is dedicated to storing world information such as energy values, IDs, signals, and the contributions of creatures per cell. Additionally, for rendering purposes, a memory area is reserved to convert the world state into RGB format for on-screen visualization. Besides this, there is a vector containing the cells of the living creatures, which helps to track and update only the active elements in the system.

Also residing on the GPU are the original model from which new creatures are generated and the vectors related to these creatures, divided into bias and weight vectors. Furthermore, a vector of variations (perturbations) relative to the original model is maintained for the new creatures generated each epoch.

To manage performance scoring, vectors are maintained that accumulate energy values and occupancy metrics held by each creature throughout the epoch.

Finally, we introduced memory areas called workspaces, the most critical and time-sensitive part of the program, used to perform all calculations related to each living cell's movement. These workspaces are organized into groups whose size is determined by the available video memory: for each workspace, two float arrays (one for inputs and one for outputs) must store as many values as there are neurons in the largest layer of the neural-network model. Partitioning the alive-cells array into workspace groups is critical for achieving maximum parallelism while respecting the hardware's memory constraints.

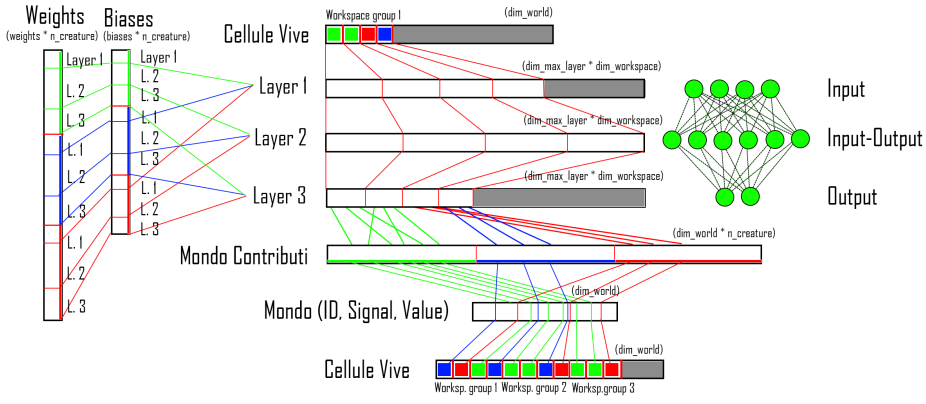


Figure 2: Memory management of the workspaces.

5 Kernel and Functions

In our project, we introduced several different CUDA kernels.

5.1 Init_curandstate

This is a very simple kernel responsible for initializing multiple curandStates. We chose to allocate only 1024 curandStates, which corresponds to the maximum number of threads per block.

5.2 Fill_random

This kernel fills a device vector with random values within a given start and end index. The generated values range between a specified minimum and maximum threshold. To speed up the generation process, it uses previously allocated curandStates.

5.3 Perlin_noise_food_and_obstacles

These two kernels perform syntactically different but closely related operations. Their purpose is to generate clusters of food or obstacles in the world, given the world values, world IDs, and a set of parameters. It is possible to modify the size and values of these clusters through input parameters such as scale, threshold, and offset. To generate the final scores and IDs, the kernels consult the `perm` vector, which defines the shape generation pattern. This vector is allocated in constant memory on the GPU to speed up access.

5.4 Add_object_to_world

This kernel works similarly to `Fill_random`, but its purpose is to add specific types of objects—such as obstacles or food—to random cells in the world. Unlike `Fill_random`, it first checks if a cell is already occupied before proceeding. If free, a random value is generated and compared against a threshold to decide if that cell will be occupied by the object type. If occupied, the cell's ID is assigned to the corresponding object type, and its value is set randomly within specified minimum and maximum thresholds.

5.5 Generate_clone_creature

This function generates variations from the original creature. The first `n` creatures under a given threshold are generated following the base model, while the subsequent creatures are generated completely randomly. The new models are stored in two continuous vectors: one for all weights and one for all biases of the newly generated models. Corresponding vectors store the variations of weights and biases compared to the original model. The curandState vector is passed as a parameter for generating random creatures.

5.6 Vision_kernel

This function populates the inputs array for every cell in the workspace group by examining each cell's neighborhood values and signals. For each living cell in the current workspace, the kernel identifies the values and signals of all cells within a surrounding window, then processes energy values so that every obstacle contributes a negative value while each cell contributes its own energy. IDs are distinguished by their signal: ally signals remain positive, whereas enemy signals are inverted.

5.7 NN_forward_weights_and_biases

This function iterates over each layer of the network in a for-loop, invoking a kernel to compute that layer's neuron values. It uses the global input, output, weights and biases arrays to combine inputs with their corresponding weights, stores the results in output, applies the biases, and then copies output back into input. This process repeats for each layer until the final layer, where output is populated with the network's actual outputs. All the cells in the workspace group process each layer simultaneously

5.8 Output_elaboration

The output of a cell consists of the first N×N values, which represent the amount of energy distributed among the N×N neighboring cells, and a final value representing its signal. While the signal value directly updates the signals array, the energy distribution populates an array called world_contribution. This array has dimensions world_total_dim * n_creature and records all energy movements in the corresponding cells, offset by each creature's ID. All the outputs of every cells in the workspace group are processed simultaneously.

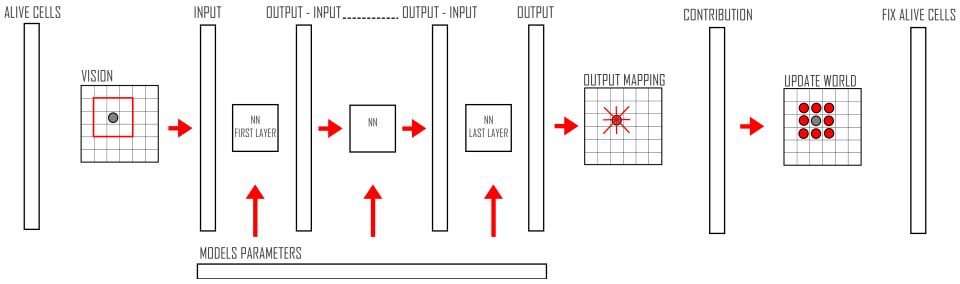


Figure 3: Simulation cycle.

5.9 World_update

After computing all the workspace groups this function updates the world state using the pre-computed contribution array. It launches one thread per world pixel; wherever there's activity, each thread separates ally and enemy energy by comparing each contribution against that pixel's original ID. Finally, it determines and records the next state of each cell.

5.10 Compact_with_thrust

This routine begins by clearing the alive-cells index array to remove any stale entries. It then scans the world-ID array, writing each living cell's index into its corresponding slot and marking dead cells with –1. Finally, it leverages Thrust's parallel compaction primitives to discard all –1 markers, yielding a dense, up-to-date list of active cell indices for the next computation cycle.

5.11 Color_map

This kernel populates the rgbWorld array (three floats per pixel) so it can be rendered in OpenGL. For each cell, it maps the cell's unique ID to a distinct hue and its energy value to brightness, producing a color-coded output in which different creatures are easily distinguished and their energy distributions visually rendered.

5.12 Compute_energy_and_occupation

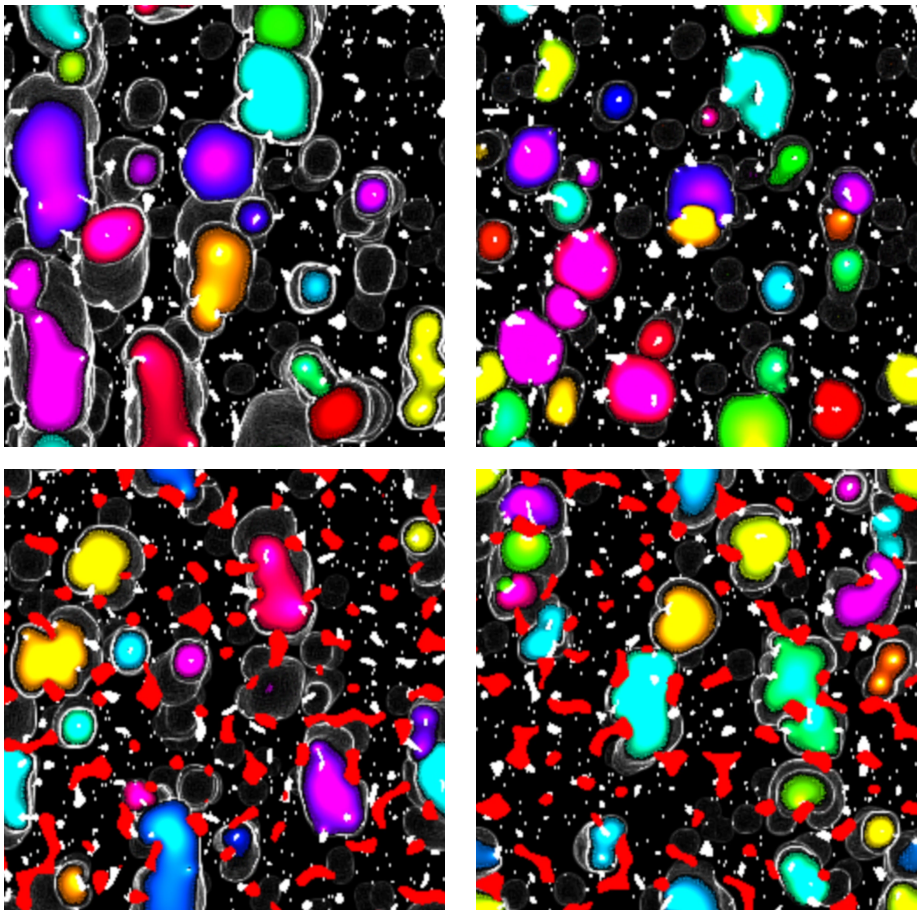
This function simply computes the cumulative scores of each creature by scanning through the `world_value` and `world_id`. The idea is to perform a cumulative sum on the score vector, where each cell corresponds to a creature. Two types of scores are calculated: occupation and cumulative energy. To reduce waiting times caused by many atomicAdd operations, we introduced a reduction method using shared memory.

5.13 Update_model

This function is responsible for updating the original model using the scores and variations of the previously generated models, following the update formula explained earlier.

To handle concurrency issues between threads, this kernel is launched with the number of parameters as the number of blocks and the number of creatures as the number of threads per block. This configuration allows for accumulating scores without worrying about race conditions.

Additionally, two values stored in shared memory are used: the total score of all creatures (which is useful for normalizing the contributions) and the sum of each creature's contributions for calculating the standard deviation.



6 Testing and Report

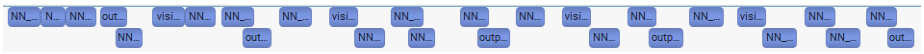
During the development of our program, we decided to experiment with different kernel setups and software configurations to determine which was the most efficient. We add on our git only the two final reports done with nsys profile. However, here we present the main changes between the different versions.

6.1 Version 1: Streams for Parallelizing Allocations

This first version featured parallelized allocation and setup (similarly for deallocation). This method was effectively usable only on the NVIDIA A5500 GPU, since it has a compute capability (cc) greater than 5, while on the 930MX only asynchronous memcpy was possible. In any case, parallelized allocation brought an actual improvement, but this gain was largely offset by the synchronization wait times.

6.2 Version 2: Streams for Computing Neural Networks

This second version used a less complex and more orderly implementation of `vision_kernel`, `NN_forward`, and `world_update`. Instead of receiving a series of workstations as input, they took a single workstation each time. To compensate for the increased number of kernel calls, we introduced streams for each workstation being processed concurrently, enabling parallel computation of each creature's output. Managing kernel calls this way made the code more readable and less verbose. However, when experimenting with the A5500, a bottleneck was observed caused by the CPU's slow kernel invocation. Consequently, even with multiple streams, effective parallelization occurred rarely and at most with two overlapping streams. Additionally, a slowdown was caused by a CUDA device synchronize call at the end of the workspace block execution, which was essential for maintaining synchronization.



6.3 Version 3: No Streams for Computing Neural Networks (Current Version)

The final version modifies the `vision_kernel`, `NN_forward`, and `world_update` kernels to operate in batch mode. All workstations are computed in a single stream (stream 0), so device synchronize calls are no longer necessary. Moreover, this approach resolves the bottleneck caused by excessive kernel calls from the CPU.



7 Limit and future Implementations

Our program still presents some limitations.

The first is the lack of scalability of the world. Specifically, if the world grows too large, it becomes impossible to run the program. Generally, simulations of such scale are not typically used. A possible solution to this issue would be to apply batching, similar to what is done in neural network computations: loading small portions of the world at a time, using padding to handle boundary issues, and then aggregating the results.

The second limitation concerns the number of creatures. The contribution matrix stored on the device scales proportionally with both the number of creatures and the size of the world. This implies that a large number of creatures combined with large worlds leads to enormous memory consumption. A possible solution could be to find a numerical encoding for the contributions per creature—for example, packing contributions within a 4- or 8-byte variable, associating each creature with a contribution scale of 100 units. However, this approach would impose maximum limits on the energy value each cell can hold.

A third limitation lies in how we create and compact the alive_cells array using Thrust reductions. As the number of living cells grows, updating this array becomes increasingly costly, and the stream synchronizations required for the reduction often dominate the runtime (outpacing both the neural-network evaluation and the world-state updates). To work around this, we tried removing the alive_cells array entirely and instead launched threads over the full world grid, simply terminating any thread whose cell is dead. This strategy delivers more stable and faster performance in large-scale simulations, but it incurs a significant overhead in smaller worlds or when relatively few cells remain active.

8 Table section

DIMENSION PARAM NAME	DESCRIPTION
world_dim (W)	indicates the side length of the world
n_creature (N)	indicates the number of creatures present in the simulation
n_layer (L)	indicates the number of layers in the model under analysis
n_weight (NW)	indicates the total number of weights of a model
n_bias (NB)	indicates the total number of biases of a model

VARIABLE NAME	DIMENSION	DESCRIPTION
world_value_h	(W, W)	world containing the actual values of each cell
world_id_h	(W, W)	world containing the corresponding IDs for each cell (-1: obstacle, 0: neutral cell, 1..N: creature IDs)
world_signal_h	(W, W)	world containing the signaling values of the cells
world_rgb_h	(W, W, 3)	world used for rendering, converts world values or world signal and world ID into an RGB map
energy_vector_h	(N)	vector containing the energy scores obtained by the various creatures
occupation_vector_h	(N)	vector containing the occupation scores of the world obtained by the various creatures
alive_cells_h	(W, W)	living cells of the world
n_cell_alive_h	1	total number of living cells (memory area allocated in shared memory)
model_weights_h	(NW)	neural network model weights
model_biases_h	(NB)	neural network model biases

VARIABLE DEVICE NAME	DIMENSION	DESCRIPTION
world_value_d	(W, W)	device counterpart of `world_value`
world_id_d	(W, W)	device counterpart of `world_id`
world_signal_d	(W, W)	device counterpart of `world_signal`
world_contributions_d	(W, W, N)	world containing the respective contributions of each creature for each cell
world_rgb_d	(H, W, 3)	device counterpart of `world_rgb`
energy_vector_d	(N)	device counterpart of `energy_vector`
occupation_vector_d	(N)	device counterpart of `occupation_vector`
model_weights_d	(NW)	device counterpart of `model_weights`
model_biases_d	(NB)	device counterpart of `model_biases`
new_models_weights_d	(NW, N)	vector containing the weights of all newly generated models
new_models_biases_d	(NB, N)	vector containing all biases of the newly generated models
variation_weights_d	(NW, N)	vector containing the weight variations of the new models compared to the starting model
variation_biases_d	(NB, N)	vector containing the bias variations of the new models compared to the starting model
alive_cells_d	(W, W)	device counterpart of `alive_cells`
n_cell_alive_d	1	device counterpart of `n_cell_alive` (memory area allocated in shared memory)
curandStates	1024	CUDA random number generator states, chosen as 1024 since it is the maximum number of threads per block
workspace_input_d	(MAX(layer_dims))	memory area dedicated to the input calculation for neural network layers
workspace_output_d	(MAX(layer_dims))	memory area dedicated to the output calculation for neural network layers

OURS GIT: GAME_OF_LIFE_V2